KubeVirt: VMs on Kubernetes

Run virtual machines on Kubernetes: architecture, install, disks and CDI, storage and live migration, networking, performance, backup, operations and moving from VMware.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Why VMs on Kubernetes and how KubeVirt works (13 min)
2. Installing KubeVirt and using virtctl (12 min)
3. VirtualMachines, VMIs and run strategies (13 min)
4. VM disks, DataVolumes and CDI (13 min)
5. Storage for VMs: access modes, block mode and snapshots (13 min)
6. VM networking: masquerade, bridge and Multus (13 min)
7. Cloud-init, instance types and VM image pipelines (13 min)
8. Live migration, node drains and eviction (13 min)
9. CPU, memory, hugepages and VM performance (13 min)
10. VM snapshots, backup and restore (13 min)
11. Operating KubeVirt: monitoring, upgrades and troubleshooting (13 min)
12. Migrating from VMware to KubeVirt (13 min)
Cheat sheet

Lesson 1. Why VMs on Kubernetes and how KubeVirt works

13 minutes. Learn why teams run virtual machines on Kubernetes and how KubeVirt turns a VM into a pod with the virt-api, virt-controller, virt-handler and virt-launcher components.

Containers share the host kernel. Some software needs its own kernel, a different operating system, or a vendor-supported VM image. KubeVirt runs those workloads on the Kubernetes you already operate.

What you will learn

- Why teams run VMs on Kubernetes
- The KubeVirt components and what each does
- How a VM becomes a pod
- What KubeVirt does not do for you

Why run VMs on Kubernetes

One platform, one set of tools. Teams move VMs from a separate hypervisor estate onto Kubernetes to share scheduling, delivery pipelines, RBAC, monitoring and networking with their containers. It suits legacy software that cannot be containerised, mixed Windows and Linux estates, and a gradual exit from a costly virtualization platform. It does not make a VM lighter than a container: each VM still carries its own guest operating system.

The components

Component	Where it runs	Job
virt-operator	Deployment	Installs and upgrades KubeVirt
virt-api	Deployment	Validates VM objects, serves console and VNC subresources
virt-controller	Deployment	Watches VMs and creates the virt-launcher pod
virt-handler	DaemonSet, one per node	Tells the node's libvirt to start, stop and migrate the domain
virt-launcher	One pod per running VM	Runs libvirt and QEMU for that single VM

Diagram: From a VirtualMachine object to a running guest

Part	What it does	What goes wrong
kubectl / virtctl	Applies a VirtualMachine object or runs virtctl start.	A wrong spec is accepted by the API but the VM may not schedule.
Kubernetes API + virt-api	Stores the VM and validates it with admission webhooks.	If virt-api is down, VM changes are rejected.
virt-controller	Creates a VirtualMachineInstance and its virt-launcher pod.	A pod stuck Pending leaves the VM in Scheduling.
virt-handler (per node)	Starts the libvirt domain on the node that was chosen.	A node without /dev/kvm cannot run the VM at full speed.
virt-launcher pod	Runs libvirt and QEMU, and holds the VM's disks and network.	Pod memory limits must include VM memory plus overhead.

Flow: kubectl / virtctl to Kubernetes API + virt-api / Kubernetes API + virt-api to virt-controller / virt-controller to virt-launcher pod / Kubernetes API + virt-api to virt-handler (per node) / virt-handler (per node) to virt-launcher pod

MARGA by EknathaLabs marga.eknathalabs.com

One VM, one pod

When you start a VirtualMachine, KubeVirt creates a VirtualMachineInstance (VMI) and a virt-launcher pod. The scheduler places the pod like any other. The pod's disks are PersistentVolumeClaims, its network comes from the CNI, and deleting the VMI deletes the pod. This is why `kubectl get pods` shows a `virt-launcher-<name>` pod for each running VM.

What it does not do

KubeVirt is not a full virtualization management suite. It does not ship backup, a cost console or a vSphere-style UI. You compose those from Kubernetes tools such as CSI snapshots, Velero and Prometheus.

Incident: The VM that stayed Pending

Incident: *A new VM sat in the Scheduling phase for an hour.*

```
$ kubectl get vmi
NAME   AGE   PHASE        IP   NODENAME
db1    58m   Scheduling
$ kubectl describe pod virt-launcher-db1-x7k2p | grep -A2 Events
Warning  FailedScheduling  0/6 nodes are available: 6 Insufficient devices.kubevirt.io/kvm.
```

The cluster nodes were virtual machines without nested virtualization, so no node offered the kvm device. The team moved the VM to bare-metal nodes with a node selector and the VMI started within a minute.

Common mistakes

- Treating a VM as lighter than a container
- Expecting a hypervisor feature to exist because the old platform had it
- Running on nodes without hardware virtualization and wondering why it is slow
- Forgetting that pod limits and VM memory are linked

Check yourself

What runs inside a virt-launcher pod?

Answer: libvirt and QEMU for one VM

Which component runs on every node as a DaemonSet?

Answer: virt-handler

What does virt-controller create for a started VM?

Answer: A VMI and a virt-launcher pod

Why might a VM stay in Scheduling?

Answer: No node offers the needed resources such as the kvm device

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Installing KubeVirt and using virtctl

12 minutes. Install the KubeVirt operator and custom resource, check hardware virtualization or emulation, and use virtctl to start VMs and open consoles.

Getting KubeVirt running is quick. Getting it running well means choosing nodes with hardware virtualization and knowing which settings are only for testing.

What you will learn

- Prerequisites and the hardware virtualization check
- Installing the operator and the KubeVirt resource
- Using virtctl day to day
- Emulation, feature gates and common install problems

Prerequisites

Nodes need hardware virtualization (Intel VT-x or AMD-V) exposed as `/dev/kvm`. Bare-metal nodes have it. Cloud VMs have it only on instance types that allow nested virtualization. A container runtime that supports the KubeVirt device plugin and a CNI are also required.

```
# On a node: is hardware virtualization available?
ls -l /dev/kvm
grep -c -E 'vmx|svm' /proc/cpuinfo
```

Install

```
export VERSION=$(curl -s https://storage.googleapis.com/kubevirt-prow/release/kubevirt/kubevirt/st \
    able.txt)
kubectl apply -f "https://github.com/kubevirt/kubevirt/releases/download/${VERSION}/kubevirt-opera \
    tor.yaml"
kubectl apply -f "https://github.com/kubevirt/kubevirt/releases/download/${VERSION}/kubevirt-cr.ya \
    ml"
kubectl -n kubevirt wait kv kubevirt --for condition=Available --timeout=10m
```

The operator manifest installs the controller and the CRD. The `KubeVirt` custom resource tells it to deploy virt-api, virt-controller and virt-handler. Pin the version in production instead of following `stable`.

Diagram: What the two manifests create

Part	What it does	What goes wrong
kubevirt-operator.yaml	Installs the operator Deployment, RBAC and the KubeVirt CRD.	Applying it alone deploys nothing else.
KubeVirt resource	Declares the install, with settings and feature gates.	A wrong setting here affects every VM.
virt-api, controller, handler	The operator deploys these components in the kubevirt namespace.	virt-handler pods only appear on schedulable nodes.
Nodes with /dev/kvm	Expose the kvm device through the device plugin.	Without it VMs fail to schedule or fall back to emulation.

Flow: kubevirt-operator.yaml to KubeVirt resource / KubeVirt resource to virt-api, controller, handler / virt-api, controller, handler to Nodes with /dev/kvm

virtctl

```
virtctl start web01
virtctl console web01
virtctl vnc web01
virtctl ssh cloud-user@web01
virtctl stop web01
```

`virtctl` is a plugin-style client that adds the VM actions kubectl cannot do. It talks to virt-api for console and VNC streams.

MARGA by EknathaLabs marga.eknathalabs.com

Emulation and feature gates

For a laptop test with no virtualization, patch the `KubeVirt` resource with `spec.configuration.developerConfiguration.useEmulation: true`. It is slow and unsupported for real work. Optional features are enabled by name under `developerConfiguration.featureGates`.

Incident: Ten times slower than expected

Incident: *Every VM on a new cluster was very slow.*

```
$ kubectl -n kubevirt get kv kubevirt -o jsonpath='{.spec.configuration.developerConfiguration}'
{"useEmulation":true}
$ ls /dev/kvm
ls: cannot access '/dev/kvm': No such file or directory
```

Someone had enabled emulation to get past a failed scheduling error on nodes without nested virtualization. The cluster was rebuilt on instance types that expose hardware virtualization and emulation was turned off.

Common mistakes

- Following the stable release URL in production
- Enabling emulation to silence a scheduling error
- Installing on nodes without hardware virtualization
- Skipping the wait for the KubeVirt resource to become Available

Check yourself

What does the KubeVirt custom resource do?

Answer: Tells the operator to deploy KubeVirt components

What file shows hardware virtualization on a node?

Answer: /dev/kvm

What is useEmulation for?

Answer: Testing without hardware virtualization

What does virtctl console do?

Answer: Opens the serial console of a VM

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. VirtualMachines, VMIs and run strategies

13 minutes. Read and write a VirtualMachine manifest, understand how it differs from a VirtualMachineInstance, and pick a run strategy such as Always or Manual.

Two objects confuse newcomers: VirtualMachine and VirtualMachineInstance. Think of a Deployment and a Pod. One is the intent, the other is the running thing.

What you will learn

- VirtualMachine versus VirtualMachineInstance
- The structure of a VM manifest
- Run strategies and what each does
- Phases and where to look when a VM does not start

VM and VMI

A **VirtualMachine** is stored in the cluster and survives stops and restarts. When it should run, KubeVirt creates a **VirtualMachineInstance** from its template. When it stops, the VMI is deleted and the VM remains. Changes to the template apply at the next start, except for a few hot-pluggable settings.

A minimal VM

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: web01
spec:
  runStrategy: Halted
  template:
    spec:
      domain:
        cpu:
          cores: 2
        memory:
          guest: 4Gi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              masquerade: {}
      networks:
        - name: default
          pod: {}
      volumes:
        - name: rootdisk
          persistentVolumeClaim:
            claimName: web01-root
```

Run strategies

Strategy	Behaviour
`Always`	Start and keep the VM running; restart it if it stops
`RerunOnFailure`	Restart only after a crash, not after a clean shutdown
`Manual`	Start and stop only on request with virtctl
`Halted`	Keep the VM stopped
`Once`	Run once and do not restart

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Lifecycle of a VirtualMachine

Part	What it does	What goes wrong
Halted	The VM object exists with no instance.	Disks and config remain; nothing runs.
Scheduling	A VMI and launcher pod are created and placed on a node.	Stuck here means no node fits, or storage is not ready.
Running	The guest is booted and the VMI has an IP.	Guest crashes are visible through the Ready condition.
Stopping	virt-handler shuts the guest down and the VMI is removed.	A force stop can corrupt the guest filesystem.

Flow: Halted to Scheduling / Scheduling to Running / Running to Stopping / Stopping to Halted

When it will not start

Read the conditions first: `kubectl get vm,vmi` and `kubectl describe vmi web01`. Pending storage, an unsatisfiable node selector and a missing image are the common causes. `kubectl get events` often names the exact reason.

Incident: A change that disappeared

Incident: *Extra memory added to a running VMI vanished after a restart.*

```
$ kubectl edit vmi web01
virtualmachineinstance.kubevirt.io/web01 edited
$ virtctl restart web01
$ kubectl get vmi web01 -o jsonpath='{.spec.domain.memory.guest}'
4Gi
```

The edit changed only the instance. On restart, KubeVirt built a new VMI from the VirtualMachine, which still said 4Gi. Editing the VirtualMachine and restarting kept the change.

Common mistakes

- Editing the VMI instead of the VM
- Using Always for a VM that must stay stopped during maintenance
- Leaving the guest memory smaller than the application needs
- Ignoring events when a VM stays in Scheduling

Check yourself

Which object is the durable definition?

Answer: VirtualMachine

Which run strategy keeps a VM stopped?

Answer: Halted

Where should a permanent memory change be made?

Answer: In the VirtualMachine template

What shows why a VM is stuck in Scheduling?

Answer: kubectl describe vmi and events

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. VM disks, DataVolumes and CDI

13 minutes. Provide VM disks with PersistentVolumeClaims, import images with the Containerized Data Importer and DataVolumes, and clone golden images.

A VM disk is a block of data presented to the guest. In KubeVirt that data lives in a PVC. CDI is the helper that gets an operating system image into that PVC.

What you will learn

- Disk and volume types
- DataVolumes and the CDI import flow
- Golden images and cloning
- Uploading your own image

Disks and volumes

In the VM spec, a **disk** is what the guest sees (bus, boot order) and a **volume** is where the data comes from. Common volume types: `persistentVolumeClaim` for durable disks, `dataVolume` for a CDI-managed PVC, `containerDisk` for a read-only image pulled from a registry (changes are lost at restart), and `cloudInitNoCloud` for first-boot configuration.

DataVolumes

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: fedora-golden
spec:
  source:
    http:
      url: https://download.example.com/fedora-cloud.qcow2
  storage:
    resources:
      requests:
        storage: 20Gi
```

CDI starts an importer pod that downloads the image, converts it to raw, and writes it into the PVC. The DataVolume goes through phases such as `ImportScheduled`, `ImportInProgress` and `Succeeded`.

Diagram: From image to VM disk

Part	What it does	What goes wrong
Image source	An http URL, a registry image or an uploaded file.	An unreachable URL leaves the import failing.
CDI importer pod	Downloads, converts to raw and writes into the PVC.	Large images need enough scratch space.
Golden PVC	The prepared base image kept in one namespace.	Changing it silently affects every later clone.
VM disk PVC	A clone created for one VM from the golden PVC.	Cloning across StorageClasses is slower.
VirtualMachine	Boots from its own PVC.	A full clone uses the full disk size.

Flow: Image source to CDI importer pod / CDI importer pod to Golden PVC / Golden PVC to VM disk PVC / VM disk PVC to VirtualMachine

Golden images and clones

Import an operating system image once into a PVC. Create each VM disk with a DataVolume whose source is that PVC. With a CSI driver that supports snapshots or clones, CDI uses a fast copy. `DataImportCron` keeps golden images fresh from a registry on a schedule.

MARGA by EknathaLabs marga.eknathalabs.com

Uploading

For a disk image on your laptop, `virtctl image-upload dv my-disk --size=20Gi --image-path=./disk.qcow2` sends it through the CDI upload proxy into a new DataVolume.

Incident: The import that never finished

Incident: *A DataVolume stayed in ImportInProgress for an hour.*

```
$ kubectl get dv fedora-golden
NAME            PHASE              PROGRESS   RESTARTS
fedora-golden   ImportInProgress   3.20%      6
$ kubectl logs importer-fedora-golden | tail -n 2
E1107 08:14:02 data-processor.go: failed to write: no space left on device
```

The scratch space for converting the qcow2 image was smaller than the image. A StorageClass with enough room for scratch data fixed it, and the import finished in a few minutes.

Common mistakes

- Downloading the same image for every VM
- Using containerDisk for data that must survive a restart
- Cloning a golden image across different storage backends without checking speed
- Not sizing the PVC larger than the image

Check yourself

What does CDI do?

Answer: Imports and clones disk images into PVCs

Which volume type loses changes at restart?

Answer: containerDisk

What keeps golden images fresh on a schedule?

Answer: DataImportCron

Why use a golden PVC?

Answer: Clones start faster than downloading again

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Storage for VMs: access modes, block mode and snapshots

13 minutes. Choose StorageClasses, access modes and volume modes for VM disks, and understand why live migration needs shared ReadWriteMany storage.

Compute is the easy part of running VMs on Kubernetes. Storage decides what you can actually do with them: migrate, snapshot, clone and survive a node failure.

What you will learn

- Access modes and why migration needs ReadWriteMany
- Block mode versus filesystem mode
- What to ask of a CSI driver
- Reading the LiveMigratable condition

Access modes

Mode	Meaning for a VM
`ReadWriteOnce`	One node at a time: a running VM cannot move without stopping
`ReadWriteMany`	Several nodes at once: required for live migration with shared storage
`ReadWriteOncePod`	One pod: not suitable for migration

During live migration the source and target virt-launcher pods both attach the disk, so the storage must allow it. Ceph RBD in block mode, many SAN CSI drivers and some NFS setups do. Plain local disks do not.

Volume mode

`volumeMode: Block` hands the raw device to QEMU, which avoids a second file system layer and usually gives better latency. `Filesystem` mode stores a disk image file in a mounted volume; it is simpler but slower and does not support RWX on most block backends. For production VMs prefer Block with an RWX capable driver.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db1-data
spec:
  accessModes: [ReadWriteMany]
  volumeMode: Block
  storageClassName: ceph-rbd
  resources:
    requests:
      storage: 100Gi
```

Diagram: Why migration needs a shared disk

Part	What it does	What goes wrong
Source node	Runs the VM and has the disk attached.	A failing node ends the VM if the disk cannot attach elsewhere.
Target node	Starts a second launcher and attaches the same disk.	With RWO the second attach is refused.
Shared RWX disk	One block device reachable from both nodes.	A slow storage network slows every VM.
CSI driver	Provides snapshots, clones and expansion.	A driver without snapshots blocks VM backup.

Flow: Source node to Shared RWX disk / Target node to Shared RWX disk / Shared RWX disk to CSI driver

MARGA by EknathaLabs marga.eknathalabs.com

What to ask of the driver

Does it support RWX Block, snapshots, cloning and online expansion? Is there a fast clone path for golden images? Can it be monitored and what is the failure mode when a node loses the storage network?

The LiveMigratable condition

KubeVirt marks each VMI with a `LiveMigratable` condition. If it is False the message says why, for example "cannot migrate VMI which does not use a shared ReadWriteMany PVC". Check it before a maintenance window, not during.

Incident: The drain that stopped every VM

> **Incident:** *Draining a node for maintenance shut down all its VMs.*

```
$ kubectl get vmi db1 -o jsonpath='{.status.conditions[?(@.type=="LiveMigratable")].message}'
cannot migrate VMI which does not use a shared ReadWriteMany PVC
$ kubectl get pvc db1-data -o jsonpath='{.spec.accessModes}'
["ReadWriteOnce"]
```

The disks were created on a ReadWriteOnce class, so KubeVirt could not migrate them during the drain. The team moved the disks to an RWX block class and rehearsed a drain before the next window.

Common mistakes

- Choosing ReadWriteOnce and expecting live migration
- Using filesystem mode for latency sensitive databases
- Not testing a drain before maintenance
- Picking a CSI driver without snapshot support

Check yourself

Which access mode lets a VM live-migrate with shared storage?

Answer: ReadWriteMany

What does Block volume mode give QEMU?

Answer: The raw device

Where do you see why a VM cannot migrate?

Answer: The LiveMigratable condition

What must a CSI driver support to snapshot VMs?

Answer: VolumeSnapshots

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. VM networking: masquerade, bridge and Multus

13 minutes. Connect VMs to the pod network with masquerade, attach them to VLANs with bridge and Multus secondary networks, and expose them with Services.

Networking is where VMs differ most from containers. Guests expect network interfaces, MAC addresses and often fixed IPs. KubeVirt maps those wishes onto Kubernetes networking.

What you will learn

- The pod network and masquerade binding
- Secondary networks with Multus and bridge
- Exposing a VM with Services
- Live migration and addresses

The pod network

Every VM can attach to the default pod network. With **masquerade** binding, the guest gets a private address and KubeVirt NATs traffic to the pod's IP. Pods and Services can then reach the VM, and the VM reaches the cluster and the internet. Masquerade also supports live migration, because the guest keeps its private address while the pod address changes. The older `bridge` binding on the pod network is limited and not recommended.

```
networks:
  - name: default
    pod: {}
  - name: vlan100
    multus:
      networkName: vlan100-bridge
domain:
  devices:
    interfaces:
      - name: default
        masquerade: {}
      - name: vlan100
        bridge: {}
```

Secondary networks

A `NetworkAttachmentDefinition` describes an extra network, for example a Linux bridge connected to VLAN 100 on the node. Multus creates the second interface. With `bridge` binding the guest sees a real L2 network and can keep its own IP or get DHCP from the VLAN. Migration across nodes then needs the same bridge and VLAN on every node.

Diagram: One VM on two networks

Part	What it does	What goes wrong
Guest OS	Sees two NICs, eth0 and eth1.	Wrong interface names in cloud-init break first boot.
Pod network (masquerade)	NATed address, reachable by Services and pods.	The guest IP is private, so inbound needs a Service.
VLAN 100 bridge	Real L2 network with its own addresses.	The bridge must exist on every node for migration.
Service / Ingress	Exposes VM ports to the cluster or outside.	A selector must match the VMI labels.
Physical network	Carries the VLAN to the data centre.	A missing trunk port blocks all VLAN traffic.

Flow: Pod network (masquerade) to Guest OS / VLAN 100 bridge to Guest OS / Service / Ingress to Pod network (masquerade) / VLAN 100 bridge to Physical network

MARGA by EknathaLabs marga.eknathalabs.com

Services and ports

`virtctl expose vm web01 --port=80 --name=web01-svc` creates a Service that selects the VMI. Network policies apply to the pod network but not to a Multus secondary interface, so secondary networks need their own controls.

Incident: The VLAN that only worked on one node

Incident: *A VM reached its VLAN, but after a migration it lost the network.*

```
$ kubectl get vmi web01 -o wide
NAME    PHASE     NODENAME
web01   Running   node3
$ ssh node3 'ip link show br100'
Device "br100" does not exist.
```

Only two of six nodes had the br100 bridge that the NetworkAttachmentDefinition named. The VM migrated to node3, which had no bridge, and lost its VLAN. The bridge was added to every node by the node configuration tool.

Common mistakes

- Using pod-network bridge binding instead of masquerade
- Forgetting the secondary bridge must exist on all nodes
- Expecting NetworkPolicy to cover Multus interfaces
- Hard-coding guest IPs on the pod network

Check yourself

Which binding is recommended for the pod network?

Answer: masquerade

What creates a secondary VM interface?

Answer: Multus with a NetworkAttachmentDefinition

Why must a VLAN bridge exist on every node?

Answer: A VM can migrate to any node

How do pods reach a VM port?

Answer: Through a Service

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Cloud-init, instance types and VM image pipelines

13 minutes. Configure a VM on first boot with cloud-init, size VMs with instance types and preferences, and build a repeatable image pipeline.

A VM you configured by hand is a pet. A VM built from an image, an instance type and a cloud-init file can be recreated at any time.

What you will learn

- Cloud-init volumes and what they can set
- Instance types and preferences
- Building and testing golden images
- Keeping secrets out of userData

Cloud-init

Most Linux cloud images run cloud-init at first boot. In KubeVirt, a `cloudInitNoCloud` volume carries the data as a small disk. Typical settings are a user, an SSH key, package installs and commands.

```
volumes:
  - name: cloudinit
    cloudInitNoCloud:
      userData: |
        #cloud-config
        users:
          - name: ops
            ssh_authorized_keys:
              - ssh-ed25519 AAAAC3Nza... ops@example
        package_update: true
        packages:
          - qemu-guest-agent
        runcmd:
          - systemctl enable --now qemu-guest-agent
```

Install `qemu-guest-agent` in every image. KubeVirt uses it for the guest IP, clean shutdown and filesystem freeze for snapshots.

Instance types and preferences

```
apiVersion: instancetype.kubevirt.io/v1beta1
kind: VirtualMachineClusterInstancetype
metadata:
  name: medium
spec:
  cpu:
    guest: 4
  memory:
    guest: 8Gi
```

A VM references it with `spec.instancetype.name: medium`. A `VirtualMachinePreference` sets device defaults such as the disk bus and network model. Changing sizes then means choosing another type, not editing CPU and memory in every VM.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: A VM assembled from reusable parts

Part	What it does	What goes wrong
Golden image	The operating system disk, built and tested once.	An untested image breaks every VM cloned from it.
Instance type	CPU and memory such as medium.	Changing it affects VMs created later, not running ones.
Preference	Disk bus, NIC model and other device defaults.	A wrong bus can stop Windows booting.
Cloud-init data	Users, keys and first-boot commands.	Secrets in userData are readable by anyone who can read the VM.
VirtualMachine	The result: a consistent, repeatable VM.	Drift returns if people log in and change it by hand.

Flow: Golden image to VirtualMachine / Instance type to VirtualMachine / Preference to VirtualMachine / Cloud-init data to VirtualMachine

Image pipelines

Build images with a tool such as Packer or a CI job: start from a vendor cloud image, apply updates, install the guest agent, test boot in a throwaway VM, then publish the disk to a registry or PVC. Version the images and keep the last few.

Secrets

Do not put passwords or tokens in `userData`. Use a Secret referenced by `userDataSecretRef`, or fetch secrets at runtime from a manager such as Vault.

Incident: Fifty different sizes

Incident: *A capacity review found VMs with 3, 5 and 7 CPUs and no pattern.*

```
$ kubectl get vm -o custom-columns=NAME:.metadata.name,CPU:.spec.template.spec.domain.cpu.cores
NAME    CPU
app01   3
app02   5
app03   7
```

Every VM was created by copying a nearby manifest and editing numbers. The team defined small, medium and large instance types, migrated VMs to them at their next restart, and sizing reviews became a single table.

Common mistakes

- Building images by logging in and installing things
- Leaving qemu-guest-agent out of the image
- Putting secrets in userData
- Expecting an instance type change to resize a running VM

Check yourself

What does cloud-init do?

Answer: Configures a VM at first boot

Why install qemu-guest-agent?

Answer: KubeVirt uses it for IP, shutdown and snapshot freeze

What is an instance type?

Answer: A reusable CPU and memory size

Where should passwords for first boot go?

Answer: In a Secret, not plain userData

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 8. Live migration, node drains and eviction

13 minutes. Move running VMs between nodes with live migration, control it with migration policies, and keep node drains and upgrades from stopping VMs.

Live migration is the feature that makes maintenance painless. It also has the most ways to fail, so it pays to know how it works and how to read its failures.

What you will learn

- How live migration works step by step
- Eviction strategy and node drains
- Migration policies and limits
- Why a migration fails or never finishes

How it works

KubeVirt starts a second virt-launcher pod on the target node and attaches the same disks. QEMU copies the guest's memory pages to the target while the VM keeps running, re-copying pages that change. When the remaining difference is small, the VM is paused for a brief moment, the last state is sent, and the target takes over.

Diagram: A live migration

Part	What it does	What goes wrong
Source virt-launcher	Runs the VM and sends memory pages.	A busy VM can change memory faster than it is copied.
Migration network	Carries the memory copy between the nodes.	A slow or shared link stretches the migration.
Shared RWX disk	Both pods attach the same disk.	RWO storage makes migration impossible.
Target virt-launcher	Receives pages and takes over after a short pause.	Missing CPU features on the target abort the move.
virt-controller	Creates the target pod and tracks progress.	Cluster limits queue extra migrations.

Flow: Source virt-launcher to Migration network / Migration network to Target virt-launcher / Source virt-launcher to Shared RWX disk / Target virt-launcher to Shared RWX disk / virt-controller to Target virt-launcher

Starting and draining

```
virtctl migrate web01
kubectl get vmim
kubectl drain node2 --ignore-daemonsets --delete-emptydir-data
```

Set `evictionStrategy: LiveMigrate` on the VM, or as a cluster default. A drain then creates migrations for VMs on the node instead of shutting them down. VMs that cannot migrate block the drain, which is safer than killing them.

Migration policies

```
apiVersion: migrations.kubevirt.io/v1alpha1
kind: MigrationPolicy
metadata:
  name: db-policy
spec:
  allowAutoConverge: true
  bandwidthPerMigration: 512Mi
  completionTimeoutPerGiB: 800
  selectors:
    virtualMachineInstanceSelector:
      workload: database
```

MARGA by EknathaLabs marga.eknathalabs.com

Auto-converge slows the guest's CPU so pages stop changing as fast. Post-copy switches early and fetches the rest on demand, which is faster but risky if the network fails. Cluster limits such as parallel migrations per node stop a drain from saturating the network.

Why they fail

Storage not shared, CPU model differing between nodes, a passthrough device (GPU) attached, a node selector that matches only one node, a network too slow for the dirty rate, or a timeout shorter than the copy needs.

Incident: The drain that would not finish

Incident: *A node drain hung for an hour with one VM left.*

```
$ kubectl get vmim -o wide
NAME           PHASE     VMI
kubevirt-m1    Running   db1
$ kubectl logs virt-launcher-db1 | tail -n 1
migration is not progressing, dirty rate is higher than transfer rate
```

The database dirtied memory faster than the 64 MiB per second bandwidth limit could copy. A policy with auto-converge and a higher limit let the migration complete in minutes.

Common mistakes

- Draining nodes before checking LiveMigratable
- Leaving a low bandwidth limit on busy VMs
- Passing through a device and expecting migration
- Running the first migration test during an outage

Check yourself

What does evictionStrategy LiveMigrate do?

Answer: Migrates VMs when a node drains

What can make a migration never finish?

Answer: Memory changing faster than it is copied

What does auto-converge do?

Answer: Slows the guest so migration can finish

What blocks migration of a VM with a GPU?

Answer: The passthrough device

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. CPU, memory, hugepages and VM performance

13 minutes. Size VM CPU and memory correctly, understand overhead and limits, and use dedicated CPUs, hugepages and NUMA for latency sensitive workloads.

Most VM performance problems come from sizing and contention, not from the hypervisor. Know what the pod limits mean, then decide whether pinning or hugepages are worth the loss of flexibility.

What you will learn

- Requests, limits and memory overhead
- Dedicated CPUs and NUMA
- Hugepages
- Overcommit and noisy neighbours

Requests, limits and overhead

The virt-launcher pod needs memory for the guest plus overhead for QEMU, libvirt and device emulation. KubeVirt adds an overhead to the pod request automatically, and it grows with CPU count and devices. A pod limit equal to guest memory causes out-of-memory kills. A VM with CPU limits can be throttled even when the node is idle.

Dedicated CPUs

```
domain:
  cpu:
    cores: 4
    dedicatedCpuPlacement: true
  resources:
    requests:
      memory: 8Gi
```

With the kubelet CPU manager set to `static` and a Guaranteed QoS pod (requests equal limits), the VM gets exclusive cores. That removes scheduling jitter from other pods. The cost is that those cores are no longer shared, so the node holds fewer VMs.

Diagram: What the VM pod needs from the node

Part	What it does	What goes wrong
Guest memory	The memory the guest operating system sees.	If the app needs more, the guest swaps or kills processes.
QEMU and libvirt overhead	Extra memory for emulation, page tables and devices.	Missing from the limit, it causes OOM kills.
Pod request and limit	The sum the scheduler and kubelet enforce.	A limit below the sum kills the container.
Dedicated CPUs	Exclusive cores from the static CPU manager.	Needs Guaranteed QoS and integer CPU counts.
Hugepages	Large memory pages reserved on the node.	Pages must be preallocated or the VM will not start.

Flow: Guest memory to Pod request and limit / QEMU and libvirt overhead to Pod request and limit / Dedicated CPUs to Pod request and limit / Hugepages to Pod request and limit

Hugepages

`memory.hugepages.pageSize: 1Gi` or `2Mi` backs guest memory with large pages, reducing translation overhead for databases and low latency software. The node must have hugepages reserved at boot or by configuration.

MARGA by EknathaLabs marga.eknathalabs.com

Overcommit and noisy neighbours

Requests lower than limits pack more VMs per node, but a busy neighbour can steal CPU. Watch CPU steal and throttling metrics, keep critical VMs on Guaranteed QoS, and separate noisy workloads with node pools.

Incident: The VM killed by its own pod

Incident: *A healthy VM restarted overnight with OOMKilled.*

```
$ kubectl describe pod virt-launcher-app01 | grep -A3 'Last State'
    Last State:  Terminated
      Reason:    OOMKilled
$ kubectl get pod virt-launcher-app01 -o jsonpath='{.spec.containers[0].resources.limits.memory}'
8Gi
```

The guest had 8 GiB and the pod limit was also 8 GiB. QEMU overhead pushed the container over its limit. Raising the pod memory limit above the guest size stopped the kills.

Common mistakes

- Setting the pod memory limit equal to the guest memory
- Pinning CPUs for every VM
- Using hugepages without reserving them first
- Ignoring CPU throttling on VMs with CPU limits

Check yourself

What must a pod limit cover for a VM?

Answer: Guest memory plus overhead

What does dedicatedCpuPlacement need?

Answer: Static CPU manager and Guaranteed QoS

What do hugepages reduce?

Answer: Memory translation overhead

What causes noisy neighbour problems?

Answer: Overcommitted shared CPU

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. VM snapshots, backup and restore

13 minutes. Take consistent VM snapshots with CSI, restore and clone them, export disks, and combine snapshots with Velero for real backups.

Backing up a VM is not the same as backing up a container. The disk holds a running operating system with open files, so how you take the copy matters as much as where you store it.

What you will learn

- Snapshots with the guest agent
- Restore and clone
- Backups that leave the cluster
- Testing a restore

Snapshots

```
apiVersion: snapshot.kubevirt.io/v1beta1
kind: VirtualMachineSnapshot
metadata:
  name: web01-before-upgrade
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: web01
```

KubeVirt creates a CSI VolumeSnapshot for each disk. With `qemu-guest-agent` running, it asks the guest to freeze its file systems briefly, so the snapshot is consistent. Without the agent, the snapshot is only crash consistent, like pulling the power. Applications such as databases need pre and post hooks for a clean state.

Restore

A `VirtualMachineRestore` object points a VM at a snapshot. The VM must be stopped, and its disks are replaced from the snapshot. To keep the original, create a new VM from the snapshot instead. The snapshot also records the VM definition, so the configuration returns with the data.

Diagram: Snapshot versus backup

Part	What it does	What goes wrong
Running VM	A guest with open files and memory state.	Copying the disk while it runs gives a torn copy.
Guest agent freeze	Pauses writes for a moment.	Without it the snapshot is crash consistent.
CSI snapshot	A fast point in time copy on the same storage.	A storage failure removes disk and snapshot together.
Backup on object storage	A copy in another place, made by Velero or an export.	An untested backup is only a hope.
Restore	Returns the VM from a snapshot or backup.	Restoring over a running VM can overwrite newer data.

Flow: Running VM to Guest agent freeze / Guest agent freeze to CSI snapshot / CSI snapshot to Backup on object storage / CSI snapshot to Restore / Backup on object storage to Restore

Backups that leave the cluster

Velero with the KubeVirt plugin backs up VM objects and, with a CSI data mover, copies volume snapshot data to object storage. `VirtualMachineExport` serves a disk over HTTP for ad hoc copies. Schedule backups, set retention, and keep at least one copy in another failure domain.

MARGA by EknathaLabs marga.eknathalabs.com

Test restores

Schedule a restore into a test namespace each month and boot the VM. Check application data, not only that the VM starts.

Incident: The backup on the same pool

Incident: *A storage pool failed and both the disks and snapshots were lost.*

```
$ kubectl get volumesnapshotcontent -o custom-columns=NAME:.metadata.name,POOL:.status.snapshotHan \
    dle
NAME             POOL
snapcontent-1   ceph-pool-a/csi-snap-77
$ ceph health
HEALTH_ERR 1 pool inconsistent
```

Every snapshot lived in the same Ceph pool as the disks it protected. The team added Velero backups to object storage in another site and a monthly restore test.

Common mistakes

- Treating snapshots as backups
- Taking snapshots without the guest agent for databases
- Never testing a restore
- Restoring over a running VM without a plan

Check yourself

What makes a VM snapshot file system consistent?

Answer: The guest agent freeze

Why is a snapshot not a backup?

Answer: It usually lives on the same storage

What is VirtualMachineRestore used for?

Answer: Returning a VM to a snapshot

What should you do with backups regularly?

Answer: Test a restore

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Operating KubeVirt: monitoring, upgrades and troubleshooting

13 minutes. Monitor KubeVirt with metrics and alerts, upgrade it and its running VMs safely, and follow a fixed order to troubleshoot a VM that will not start or perform.

A KubeVirt cluster has two layers to watch: the control plane that runs the VMs and the guests themselves. Knowing where to look first saves hours.

What you will learn

- Metrics and alerts
- Upgrading the operator and the running VMs
- A fixed troubleshooting order
- Day two checklist

Monitoring

KubeVirt exposes Prometheus metrics for its components and for each VMI, for example vCPU time, memory usage and network traffic. Alert on virt-api or virt-controller down, virt-handler not ready on a node, failed or stuck migrations, VMs stuck in Scheduling, and guests with sustained CPU throttling. Also watch the storage layer, since it is the usual source of slow VMs.

Upgrades

Apply a newer operator manifest and the KubeVirt resource follows. The control plane components roll. Running VMs keep their old virt-launcher until restarted or migrated. Set `workloadUpdateStrategy.workloadUpdateMethods: [LiveMigrate]` so KubeVirt migrates VMs to launchers of the new version in batches. VMs that cannot migrate need a planned restart.

Diagram: Upgrading KubeVirt without stopping VMs

Part	What it does	What goes wrong
New operator	Rolls the control plane to the new version.	Skipping versions can break the upgrade path.
New control plane	virt-api, virt-controller and virt-handler updated.	Check all pods are Ready before the next step.
VMs on old launchers	Still running the previous launcher version.	They stay old until restarted or migrated.
Batch live migration	Moves VMs to new launchers a few at a time.	Non migratable VMs block and need a restart.
VMs on new launchers	The goal state with the new version everywhere.	Verify with the launcher image on each VMI.

Flow: New operator to New control plane / New control plane to VMs on old launchers / New control plane to Batch live migration / VMs on old launchers to Batch live migration / Batch live migration to VMs on new launchers

Troubleshooting order

1. `kubectl get vm,vmi`: phase and conditions.
2. `kubectl describe vmi` and events: reasons for Pending or failed.
3. The virt-launcher pod: scheduling, image pull, OOMKilled.
4. The node and virt-handler logs.
5. Storage: PVC bound, DataVolume phase, CSI health.
6. Network: bindings, Multus, Service selectors.

MARGA by EknathaLabs marga.eknathalabs.com

7 The guest: `virtctl console`, guest logs, guest agent.

Day two checklist

Alerts in place, a tested drain, a tested restore, images versioned, upgrade rehearsed in a test cluster, and a record of which VMs cannot migrate.

Incident: The upgrade that left old launchers

Incident: *Weeks after an upgrade, VMs still ran the old launcher image.*

```
$ kubectl get vmi -o custom-columns=NAME:.metadata.name,LAUNCHER:.status.launcherContainerImageVer \
    sion
NAME    LAUNCHER
web01   quay.io/kubevirt/virt-launcher:v1.2.0
$ kubectl get kv -n kubevirt kubevirt -o jsonpath='{.status.observedKubeVirtVersion}'
v1.3.0
```

The control plane was on v1.3.0, but the VMs still used v1.2.0 launchers because no workload update method was set. Enabling LiveMigrate moved them over in a weekend window.

Common mistakes

- Assuming an operator upgrade upgrades the VMs
- Skipping alerts on stuck migrations
- Troubleshooting the guest before checking storage and scheduling
- Upgrading production first

Check yourself

What does an operator upgrade not change by itself?

Answer: Running virt-launcher versions

Which setting moves VMs to new launchers?

Answer: workloadUpdateStrategy with LiveMigrate

Where do you start troubleshooting a VM?

Answer: kubectl get vm,vmi and events

What often causes slow VMs?

Answer: The storage layer

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Migrating from VMware to KubeVirt

13 minutes. Plan a move from VMware to KubeVirt: inventory, conversion tools such as the Migration Toolkit for Virtualization, driver and network changes, and what has no direct equivalent.

Many teams look at KubeVirt because of licensing changes on their current hypervisor. The technology move is manageable. The harder part is the people, process and tooling around the VMs.

What you will learn

- Inventory and migration waves
- Tools that convert VMs
- Drivers, networks and storage mapping
- What has no direct equivalent

Inventory and waves

List every VM with its operating system, size, disks, networks, dependencies, owner and downtime tolerance. Group them in waves: start with low risk Linux VMs, then application tiers, and leave the hardest (clustered databases, special hardware, old operating systems) for last. Each wave has a cutover plan, a test and a rollback.

Conversion

The **Migration Toolkit for Virtualization** (upstream: Forklift) connects to vCenter, reads VMs and converts them into KubeVirt VirtualMachines. It uses `virt-v2v` to adapt the guest: it installs virtio drivers, removes VMware tools hooks and rewrites boot configuration. It supports a cold migration (VM off during the copy) and a warm migration (copy while running, then a short final sync).

Diagram: A migration plan, wave by wave

Part	What it does	What goes wrong
Inventory	Every VM, its dependencies and its owner.	Missing a dependency breaks the application at cutover.
Migration plan	Maps networks, storage and waves.	A wrong network map isolates the VM.
Conversion (Forklift)	Copies disks and adapts the guest with virt-v2v.	A missing driver stops the guest booting.
Test in the cluster	Boots the VM, checks the application and performance.	Skipping the test moves the failure to cutover.
Cutover and rollback	Switches users to the new VM and keeps the old one for a while.	Deleting the source too early removes the rollback.

Flow: Inventory to Migration plan / Migration plan to Conversion (Forklift) / Conversion (Forklift) to Test in the cluster / Test in the cluster to Cutover and rollback

Drivers, networks and storage

Linux guests usually have virtio drivers already. Windows guests need them installed, and boot problems usually mean a missing storage driver. Map each vSphere port group to a bridge or Multus network, and each datastore to a StorageClass. Replace VMware Tools with the qemu guest agent. Re-check IP addresses, MAC address based licences and monitoring agents.

No direct equivalent

Distributed resource scheduling becomes the Kubernetes scheduler plus a descheduler. Backup moves from a VMware API product to CSI snapshots and Velero. Network virtualization such as NSX becomes network policies and CNI features. Plan training and runbooks, because operators will use kubectl instead of a vSphere client.

MARGA by EknathaLabs marga.eknathalabs.com

Incident: The Windows VMs that would not boot

Incident: *Converted Windows VMs showed a blue screen after the first start.*

```
$ virtctl vnc win01
STOP: 0x0000007B INACCESSIBLE_BOOT_DEVICE
$ kubectl get vm win01 -o jsonpath='{.spec.template.spec.domain.devices.disks[0].disk.bus}'
virtio
```

The guests were converted to a virtio disk bus but no virtio storage driver was installed. The team installed the driver in the source VM before conversion, or temporarily used a SATA bus, then switched to virtio.

Common mistakes

- Migrating the largest VMs first
- Deleting source VMs right after cutover
- Forgetting drivers for Windows guests
- Planning only the VMs and not the operations around them

Check yourself

Which tool upstream of the Migration Toolkit converts VMs?

Answer: Forklift

What does a warm migration do?

Answer: Copies while the VM runs, then a short final sync

What commonly stops a converted Windows VM booting?

Answer: A missing virtio storage driver

How should migrations be ordered?

Answer: Low risk waves first, hardest last

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: KubeVirt

Install, VM objects, disks, networking, live migration, snapshots and a troubleshooting order for VMs on Kubernetes.

Check and install

```
ls -l /dev/kvm
kubectl apply -f kubevirt-operator.yaml
kubectl apply -f kubevirt-cr.yaml
kubectl -n kubevirt wait kv kubevirt --for condition=Available --timeout=10m
```

Warning: `useEmulation: true` *runs VMs in software and is for tests only. Real workloads need* `/dev/kvm`.

Day-to-day with virtctl

```
virtctl start web01
virtctl stop web01
virtctl console web01
virtctl vnc web01
virtctl ssh cloud-user@web01
virtctl expose vm web01 --port=80 --name=web01-svc
```

Warning: *Edit the VirtualMachine, not the VMI. Changes to a VMI vanish at the next start.*

Objects

Object	Meaning
VirtualMachine	Durable definition
VirtualMachineInstance	One running copy
DataVolume	Disk created and filled by CDI
VirtualMachineSnapshot	Point in time copy of disks and definition
MigrationPolicy	Rules for bandwidth and timeouts

Disks

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: golden
spec:
  source:
    http:
      url: https://example.com/image.qcow2
  storage:
    resources:
      requests:
        storage: 20Gi
```

Danger: `containerDisk` *is ephemeral. Data written to it is lost when the VM restarts.*

Migration

```
virtctl migrate web01
kubectl get vmim
kubectl get vmi web01 -o jsonpath='{.status.conditions[?(@.type=="LiveMigratable")].message}'
```

Warning: *ReadWriteOnce disks and passthrough devices block live migration. Check LiveMigratable before a drain.*

Snapshots

```
kubectl apply -f snapshot.yaml
kubectl get vmsnapshot
```

Warning: *A snapshot on the same storage is not a backup. Copy data off the cluster with Velero or an export.*

MARGA by EknathaLabs marga.eknathalabs.com

Troubleshooting order

Step	Look at
1	`kubectl get vm,vmi` phase and conditions
2	`kubectl describe vmi` and events
3	The virt-launcher pod: scheduling, OOMKilled
4	Node and virt-handler logs
5	Storage: PVC bound, DataVolume phase
6	Network: binding, Multus, Service selector
7	The guest: console, guest agent

Quick health sweep

```
kubectl get kubevirt -n kubevirt
kubectl get vmi -A
kubectl get dv,pvc -A
kubectl get vmim -A
virtctl console VMNAME
```

MARGA by EknathaLabs
marga.eknathalabs.com